

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Математические основы интеллектуальных систем»
на тему:

**БАЗА ЗНАНИЙ ИНТЕЛЛЕКТУАЛЬНОГО
ПОМОЩНИКА ПО ПОДГОТОВКЕ К
ТЕХНИЧЕСКИМ СОБЕСЕДОВАНИЯМ**

БГУИР КП4 6-05-0611-03 026 ПЗ

Студент гр. 421702
Проверяющий

Н. В. Сечейко
Д. А. Сальников

Минск 2026

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет
Специальность

ИТиУ
6-05-0611-03

Кафедра

ИИТ

УТВЕРЖДАЮ

_____ Зав. кафедрой
«____» _____ 2026 г.

ЗАДАНИЕ

по курсовому проекту студента

Сечейко Николая Владимировича

(фамилия, имя, отчество)

1. Тема работы: База знаний интеллектуального помощника по подготовке к
техническим собеседованиям

2. Срок сдачи студентом законченной работы: 11.05.2026

3. Исходные данные к проекту: _____

4. Содержание пояснительной записки (перечень подлежащих разработке вопросов):

Введение

1 Анализ подходов к разработке

2 Проектирование

3 Разработка

Заключение

5. Перечень графического материала (с точным указанием обязательных чертежей):
Компьютерная презентация курсового проекта _____

КАЛЕНДАРНЫЙ ПЛАН

№ п/п	Наименование этапов курсовой работы	Объем этапа, %	Срок выполнения этапов	Примечание
1	Подбор и изучение литературы	10	9.02 – 20.02	
2	Изучение проблемной области, средств проектирования и разра- ботки	10	21.02 – 27.03	
3	Определение требований к базе знаний системы	10	28.02 – 06.03	
4	Проектирование модели базы знаний системы	25	07.03 – 13.04	
5	Реализация базы знаний системы	30	14.04 – 08.05	
6	Оформление пояснительной за- писки	10	01.05 – 11.05	
7	Оформление графической части проекта	5	06.05 – 11.05	

Дата выдачи задания: 20.02.2026 г. Проверяющий _____ И. О. Фамилия

Задание принял к исполнению _____ И. О. Фамилия

СОДЕРЖАНИЕ

Перечень условных обозначений	4
Введение	5
1 Анализ предметной области и требований к базе знаний	8
1.1 Характеристика предметной области	8
1.2 Типы задач и сценарии использования	11
1.3 Анализ существующих решений и баз знаний	14
1.4 Анализ требований к базе знаний	17
1.5 Вывод по разделу анализа	19
2 Проектирование базы знаний интеллектуальной системы	21
2.1 Общая архитектура интеллектуальной системы и место базы знаний	21
2.2 Выбор формализма и логической основы	23
2.3 Концептуальная модель предметной области	25
2.4 Логическая и физическая структура базы знаний	27
2.5 Требования к программному прототипу (MVP)	29
2.6 Обеспечение согласованности, расширяемости и сопровождаемости	33
2.7 Вывод по разделу проектирования	34
3 Разработка базы знаний и инструментов для работы с ней	37
3.1 Выбор технологических средств реализации	37
3.2 Реализация структуры и наполнения базы знаний	38
3.3 Реализация механизмов доступа к базе знаний и её использования	40
3.4 Демонстрация функционирования базы знаний	41
3.5 Подход к тестированию	42
3.6 Тестовый набор и его классификация	43
3.7 Представление и результаты тестирования	43
3.8 Оценка качества базы знаний	44
3.9 Итоги раздела, посвящённого реализации	45
Заключение	46
Список использованных источников	48

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

1. API – Application Programming Interface (интерфейс программирования приложений);
2. ASGI – Asynchronous Server Gateway Interface (асинхронный шлюзовой интерфейс сервера);
3. BFS/DFS – Breadth-First/Depth-First Search (алгоритмы обхода графа в ширину/в глубину);
4. БЗ – База знаний;
5. CI/CD – Continuous Integration/Continuous Delivery (непрерывная интеграция и доставка);
6. CLI – Command Line Interface (интерфейс командной строки);
7. CS – Computer Science (фундаментальные основы информатики);
8. EdTech – Educational Technology (образовательные технологии);
9. FAANG – Facebook, Amazon, Apple, Netflix, Google (ведущие технологические компании);
10. Git – распределённая система контроля версий;
11. HR-tech – Human Resources Technology (технологии управления персоналом);
12. JSON – JavaScript Object Notation (формат обмена данными);
13. LRU – Least Recently Used (алгоритм вытеснения наименее недавно использованных данных);
14. MVP – Minimum Viable Product (минимально жизнеспособный продукт);
15. OOP – Object-Oriented Programming (объектно-ориентированное программирование);
16. PR – Pull Request (запрос на слияние изменений в репозиторий);
17. RAG – Retrieval-Augmented Generation (генерация с дополненным поиском);
18. REST – Representational State Transfer (архитектурный стиль проектирования API);
19. SLA/SLO/SLI – Service Level Agreement/Objective/Indicator (соглашение/цель/индикатор уровня обслуживания);
20. SPA – Single Page Application (одностраничное приложение);
21. STAR – Situation, Task, Action, Result (методика структурирования поведенческих ответов);
22. TechInterview – интеллектуальный помощник по подготовке к техническим собеседованиям;
23. TTL – Time To Live (время жизни записи или сессии);
24. UI/UX – User Interface/User Experience (пользовательский интерфейс/пользовательский опыт);

ВВЕДЕНИЕ

Актуальность темы базы знаний

Актуальность разработки базы знаний (БЗ) для интеллектуальной системы подготовки к техническим собеседованиям обусловлена растущими требованиями к фундаментальной подготовке специалистов в области информационных технологий. В условиях глобальной конкуренции на рынке труда, особенно при отборе в ведущие технологические компании (FAANG, Yandex, JetBrains и др.), кандидатам необходимо демонстрировать глубокое понимание базовых концепций Computer Science: алгоритмов и структур данных, принципов работы операционных систем, сетевых протоколов, моделей данных и методов параллельного программирования.

Проект реализуется в контексте дисциплины «Математические основы интеллектуальных систем» и опирается на результаты предыдущих учебных этапов по инженерии знаний, онтологическому моделированию и семантическим технологиям. Разрабатываемая БЗ решает задачи интеллектуальной поддержки: *диагностики* уровня владения концепциями, *классификации* учебных материалов по доменам и уровням сложности, *формирования адаптивных траекторий* подготовки и *генерации объяснений* с варьированием глубины изложения. Целевая аудитория включает кандидатов уровней Junior–Senior, специалистов, осуществляющих карьерный переход, а также технических менторов.

Цель и задачи курсового проекта

Целью курсового проекта является разработка структурированной базы знаний фундаментальных основ Computer Science, выступающей семантическим ядром интеллектуального помощника по подготовке к техническим собеседованиям.

Для достижения поставленной цели сформулированы следующие задачи:

1. провести анализ предметной области «Фундаментальные основы Computer Science», выделив ключевые концепции, их иерархические и семантические взаимосвязи, а также типовые сценарии применения знаний в интервью;
2. обосновать выбор формализма представления знаний (онтологическая модель, метаданные, таксономия тегов) и сформулировать требования к машиночитаемости, расширяемости и согласованности БЗ;
3. спроектировать логическую и физическую структуру базы знаний, обеспечивающую эффективную навигацию, перекрёстные ссылки и поддержку различных типов единиц знаний (концепции, задачи, паттерны,

шпаргалки);

4. реализовать прототип наполнения БЗ типовыми материалами по алгоритмам, структурам данных и системным концепциям, продемонстрировав механизмы доступа и использования в сценариях подготовки;

5. выполнить тестирование корректности семантических связей и полноты покрытия ключевых тем, а также оценить качество разработанной структуры с точки зрения сопровождаемости.

Объект, предмет и границы разработки

Объектом разработки является подсистема управления знаниями в составе интеллектуальной системы «Интеллектуальный помощник по подготовке к техническим собеседованиям» (TechInterview), отвечающая за хранение, организацию и предоставление доступа к предметно-ориентированным материалам.

Предметом проектирования выступает база знаний фундаментальных основ Computer Science: виды представляемых знаний (декларативные концепции, процедурные паттерны решения задач, мета-знания о сложности и связях), форматы представления (Markdown с YAML-фронтматтером), логическая модель с доменной таксономией и механизм семантической навигации.

Границы проекта определяют функциональную область: поддержка задач диагностики знаний, классификации материалов по темам и уровням, статической генерации учебных последовательностей на основе правил. Входными данными служат структурированные карточки знаний и профили пользователей с указанием целевой роли и уровня. За рамками проекта остаются: динамическая адаптация на основе ML-моделей, полноценный пользовательский интерфейс, интеграция с внешними платформами для проведения собеседований в реальном времени.

Структура пояснительной записки

Пояснительная записка состоит из введения, трёх основных разделов и заключения.

– *Раздел 1* посвящён аналитическому исследованию: проводится характеристика предметной области фундаментальных основ Computer Science, анализируются типовые сценарии использования, рассматриваются существующие образовательные ресурсы и формулируются функциональные и нефункциональные требования к базе знаний.

– *Раздел 2* описывает проектные решения: обосновывается выбор онтологического формализма, проектируется концептуальная модель предметной области, определяются логическая и физическая структуры БЗ,

система метаданных и правила обеспечения согласованности знаний.

– *Раздел 3* посвящён практической реализации: описываются инструменты и форматы реализации, процесс наполнения базы знаний типовыми единицами (карточки алгоритмов, задачи на структуры данных, паттерны решения), демонстрируются сценарии использования и приводятся результаты тестирования корректности связей.

– *В заключении* формулируются выводы о достижении цели проекта, оценивается качество разработанной базы знаний с точки зрения полноты, непротиворечивости и расширяемости, а также определяются направления дальнейшего развития системы в части интеграции с модулями адаптивного обучения и генерации персонализированных сценариев собеседований.

Курсовой проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности соответствует норме, установленной кафедрой ИИТ. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников».

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ К БАЗЕ ЗНАНИЙ

1.1 Характеристика предметной области

Предметная область курсового проекта — «*Фундаментальные основы Computer Science*» — представляет собой систематизированное множество базовых концепций, принципов, алгоритмов и паттернов, составляющих теоретический и практический базис для успешного прохождения технических собеседований в сфере разработки программного обеспечения. Данная область является первым уровнем (*Level 1*) в трёхуровневой архитектуре базы знаний интеллектуального помощника *TechInterview* и служит фундаментом для двух других предметных областей: «Проектирование и архитектура систем» и «Процесс собеседования и soft skills».

Типы единиц знаний предметной области

Для представления знаний в базе знаний определены шесть основных типов единиц информации, каждый из которых выполняет специфическую роль в процессе подготовки к собеседованиям. Типы единиц знаний, их коды и описания приведены в таблице 1.1.

Таблица 1.1 – Типы единиц знаний, специфичных для предметной области

Тип	Код	Описание
Карточка концепции	CONCEPT	Теоретическая концепция CS (структура данных, алгоритм, принцип)
Карточка задачи	PROBLEM	Алгоритмическая задача с решением и разбором
Паттерн решения	PATTERN	Типовой подход к классу задач
Карточка вопроса	QUESTION	Вопрос, задаваемый на собеседовании, с эталонным ответом
Шпаргалка	CHEATSHEET	Компактная сводка по теме для быстрого повторения
Типичная ошибка	PITFALL	Распространённая ошибка кандидата с объяснением

Каждый тип единиц знаний характеризуется собственным набором атрибутов, метаданных и семантических связей, что будет детально описано в разделе 2 при проектировании логической структуры базы знаний.

Основные понятия, объекты и отношения

Ключевыми *объектами* предметной области являются:

- **Концепции (CONCEPT)** — абстрактные или конкретные сущности, описывающие фундаментальные идеи Computer Science (например, «Двоичное дерево поиска», «Семафор», «Принцип инкапсуляции», «Протокол TCP»). Каждая концепция характеризуется определением, областью применения, сложностными оценками и связями с другими концепциями.

- **Алгоритмы и структуры данных** — процедурные знания, описывающие способы организации и обработки информации (сортировки, обходы графов, хеширование, динамическое программирование). Для каждого алгоритма фиксируются: вход/выход, асимптотическая сложность, инварианты, типовые варианты реализации.

- **Паттерны решения задач (PATTERN)** — обобщённые стратегии применения знаний в контексте интервью (например, «Two Pointers», «Sliding Window», «DFS/BFS template»). Паттерны связывают концепции с типовыми формулировками задач и приёмами их декомпозиции.

- **Задачи (PROBLEM)** — алгоритмические задачи с условиями, решениями на различных языках программирования, тестовыми примерами и подробными разборами. Задачи классифицируются по домену, уровню сложности (Junior/Middle/Senior) и типу.

- **Вопросы (QUESTION)** — единицы проверки теоретических знаний, включающие формулировку вопроса, эталонный ответ, критерии оценки и ссылки на релевантные концепции.

- **Шпаргалки (CHEATSHEET)** — компактные справочные материалы, содержащие ключевые формулы, определения, шаблоны кода и мнемонические правила для быстрого повторения перед собеседованием.

- **Типичные ошибки (PITFALL)** — описания распространённых заблуждений и ошибок кандидатов (например, «Путаница между стеклом и кучей», «Некорректная оценка сложности вложенных циклов») с пояснениями, как их избежать.

- **Компетенции** — абстрактные метки, отражающие уровень владения тем или иным разделом (например, «Понимает асимптотический анализ», «Умеет реализовывать обход дерева»). Компетенции используются для диагностики и формирования учебных траекторий.

Отношения между объектами формируют семантическую сеть предметной области и включают:

- **Таксономические связи (is-a, part-of)**: иерархия концепций (например, «Красно-чёрное дерево» is-a «Самобалансирующееся дерево»);

- **Пререквизиты (requires)**: зависимость понимания одной концепции от другой (например, для изучения «Динамического программирования» необходимо знать «Рекурсию» и «Мемоизацию»);

- **Применение (used-in)**: связь алгоритма/паттерна с типами задач или доменами (например, «*BFS*» used-in «*Задачи на поиск кратчайшего пути*»);
- **Оценка сложности (has-complexity)**: привязка асимптотических характеристик к алгоритмам и структурам данных;
- **Уровень освоения (target-level)**: соответствие концепции или задачи определённому уровню кандидата (Junior/Middle/Senior);
- **Проверка компетенции (assesses)**: связь вопроса/задачи с компетенциями, которые она проверяет.

Ограничения и особенности предметной области

При разработке базы знаний необходимо учитывать следующие *ограничения и особенности*:

- **Объём и динамика знаний**: предметная область характеризуется большим количеством концепций (сотни единиц) и умеренной скоростью обновления (появление новых паттернов, изменение требований компаний). База знаний должна поддерживать инкрементальное пополнение без нарушения целостности.
- **Требования к точности**: знания в области Computer Science требуют высокой формальной корректности (особенно в части асимптотического анализа, инвариантов алгоритмов, спецификаций протоколов). Недопустимы противоречивые определения или некорректные оценки сложности.
- **Требования к времени ответа**: в интерактивных сценариях (поиск, диагностика, генерация объяснений) время формирования ответа не должно превышать 1–2 секунд для сохранения качества пользовательского опыта.
- **Источники знаний**: основными источниками являются академические учебники, документация технических компаний, материалы открытых образовательных платформ, а также экспертные оценки технических интервьюеров. Все знания должны проходить верификацию и сопровождаться ссылками на авторитетные источники.
- **Мультиязычность и вариативность формулировок**: одни и те же концепции могут называться по-разному в различных источниках (например, «*hash table*» / «*hash map*» / «*ассоциативный массив*»). База знаний должна поддерживать синонимию и нормализацию терминов.
- **Контекстная зависимость**: сложность и глубина изложения материала должны адаптироваться под роль пользователя (кандидат/ментор) и целевую компанию (стартап/FAANG).

Таким образом, предметная область «Фундаментальные основы Computer Science» представляет собой структурируемое, но объёмное мно-

жестов взаимосвязанных концепций, требующее применения формальных методов представления знаний для обеспечения точности, согласованности и эффективности использования в интеллектуальной системе подготовки к собеседованиям.

1.2 Типы задач и сценарии использования

Разрабатываемая база знаний фундаментальных основ Computer Science предназначена для поддержки широкого спектра интеллектуальных задач в контексте подготовки к техническим собеседованиям. Ниже приведена систематизация типов задач, решаемых с использованием БЗ, а также детализированные сценарии использования (use-cases), демонстрирующие взаимодействие пользователей и внешних модулей системы с базой знаний.

Классификация типов задач

На основе анализа предметной области и требований целевой аудитории выделены следующие основные типы задач, для решения которых предназначена база знаний:

1. **Диагностика уровня знаний** — определение степени владения пользователем ключевыми концепциями и компетенциями на основе анализа ответов на вопросы и решения задач.

2. **Классификация материалов** — автоматическая категоризация единиц знаний по доменам (алгоритмы, структуры данных, ОС, сети, БД), уровням сложности (Junior/Middle/Senior) и типам (теория, задача, паттерн).

3. **Поиск и выбор релевантных знаний** — семантический поиск концепций, задач и объяснений по запросам пользователя с учётом контекста (роль, уровень, целевая компания).

4. **Планирование учебной траектории** — формирование персонализированной последовательности изучения тем на основе выявленных пробелов в знаниях и целевых требований позиции.

5. **Объяснение концепций** — предоставление определений, примеров, визуализаций и связей с другими концепциями с варьированием глубины изложения.

6. **Генерация вопросов и задач** — создание типовых формулировок интервью-вопросов и задач на код на основе шаблонов, хранящихся в БЗ.

Каждый тип задач опирается на определённые виды знаний и механизмы доступа к ним, что будет детально описано в разделе 2 при проектировании формализма представления знаний.

Сценарии использования (Use-Cases)

Для демонстрации практического применения базы знаний сформулированы четыре реалистичных сценария использования, охватывающих основные роли пользователей системы и типы взаимодействий.

Сценарий UC-1: Диагностика уровня подготовки кандидата

- **Актор:** Кандидат (Junior/Middle/Senior уровень).
- **Входные данные:** Профиль пользователя (целевая роль, желаемая компания), ответы на серию диагностических вопросов.
- **Обращение к БЗ:** Модуль диагностики запрашивает из БЗ вопросы по указанным доменам, соответствующие уровню кандидата, а также эталонные ответы и критерии оценки.
- **Обработка:** Система сопоставляет ответы пользователя с эталонами, вычисляет процент правильных ответов по каждой теме, определяет пробелы в знаниях.
- **Выходные данные:** Профиль компетенций с оценкой по каждому домену, список рекомендованных тем для изучения, визуализация прогресса.
- **Связанные типы задач:** Диагностика, классификация, планирование.

Сценарий UC-2: Персонализированная рекомендация материалов для подготовки

- **Актор:** Кандидат, осуществляющий подготовку к собеседованию.
- **Входные данные:** Результаты диагностики, целевая позиция (например, «*Backend Developer, Middle, Yandex*»), доступное время на подготовку.
- **Обращение к БЗ:** Модуль рекомендаций запрашивает концепции и задачи, связанные с выявленными пробелами, фильтрует их по уровню сложности и домену, учитывает пререквизиты.
- **Обработка:** Система формирует упорядоченную последовательность изучения с учётом зависимостей между темами (например, перед «*Динамическим программированием*» рекомендуется изучить «*Рекурсию*» и «*Мемоизацию*»).
- **Выходные данные:** Учебный план на 2–4 недели с ежедневными задачами, ссылками на материалы и оценками времени на изучение.
- **Связанные типы задач:** Планирование, поиск, классификация.

Сценарий UC-3: Объяснение концепции с адаптивной глубиной

- **Актор:** Кандидат или ментор в процессе обучения.

- **Входные данные:** Запрос на объяснение концепции (например, «Объясни, как работает хеш-таблица»), указание желаемого уровня детализации (базовый/продвинутой).
- **Обращение к БЗ:** Модуль объяснений извлекает из БЗ карточку концепции с определением, примерами реализации, оценками сложности, связанными паттернами и типовыми вопросами.
- **Обработка:** Система адаптирует формулировки под уровень пользователя, выбирает релевантные примеры, формирует связный текст с перекрёстными ссылками.
- **Выходные данные:** Структурированное объяснение с разделами: определение, принцип работы, пример кода, сложность, связанные концепции, типовые вопросы на собеседовании.
- **Связанные типы задач:** Объяснение, поиск, классификация.

Сценарий УС-4: Генерация интервью-сессии для ментора

- **Актор:** Технический интервьюер или ментор.
- **Входные данные:** Параметры сессии (уровень кандидата, длительность, фокус-домены, компания-ориентир).
- **Обращение к БЗ:** Модуль генерации запрашивает из БЗ набор вопросов и задач, соответствующих указанным критериям, включая критерии оценки и подсказки для интервьюера.
- **Обработка:** Система комбинирует вопросы разных типов (теория, задача на код, системный дизайн), обеспечивает покрытие ключевых тем, балансирует сложность.
- **Выходные данные:** Готовый сценарий интервью на 45–60 минут с таймингом, вопросами, ожидаемыми ответами и рекомендациями по оценке.
- **Связанные типы задач:** Генерация, классификация, поиск.

Связь сценариев с архитектурой системы

Описанные сценарии использования определяют требования к интерфейсам доступа к базе знаний и механизмам обработки запросов. В таблице 1.2 приведено обобщённое соответствие сценариев, типов задач и задействованных компонентов системы.

Ожидаемые результаты использования

Реализация описанных сценариев с использованием разрабатываемой базы знаний должна обеспечить следующие результаты:

- **Для кандидатов:** сокращение времени подготовки на 30–40% за счёт персонализации, повышение уверенности за счёт структурированного покрытия тем, объективная оценка готовности к собеседованию.

Таблица 1.2 – Соответствие сценариев использования, типов задач и компонентов системы

Сценарий	Типы задач	Задействованные компоненты
УС-1: Диагностика	Диагностика, классификация	Модуль диагностики, БЗ вопросов
УС-2: Рекомендации	Планирование, поиск	Модуль рекомендаций, БЗ концепций
УС-3: Объяснение	Объяснение, поиск	Модуль объяснений, БЗ концепций
УС-4: Генерация интервью	Генерация, классификация	Модуль генерации, БЗ задач

– **Для менторов и интервьюеров:** стандартизация процесса проведения технических интервью, снижение нагрузки на подготовку материалов, повышение согласованности оценки кандидатов.

– **Для системы в целом:** масштабируемая основа для расширения предметных областей (System Design, Soft Skills), возможность интеграции с внешними образовательными платформами, поддержка аналитики эффективности подготовки.

Таким образом, сценарии использования демонстрируют практическую ценность базы знаний как центрального компонента интеллектуальной системы подготовки, обеспечивающего решение ключевых задач диагностики, рекомендаций, объяснения и генерации материалов для технических собеседований.

1.3 Анализ существующих решений и баз знаний

Для обоснования архитектурных и формальных решений при разработке базы знаний «Фундаментальные основы Computer Science» был проведён сравнительный анализ существующих платформ и систем, решающих сходные задачи представления знаний и поддержки подготовки к техническим собеседованиям.

Обзор аналогичных систем

В качестве аналогов выбраны три системы, представляющие различные парадигмы организации знаний: платформа оценки навыков, академическая онтологическая система и визуально-ориентированный образовательный ресурс.

HackerRank Назначение: Платформа для оценки технических компетенций и проведения скрининга кандидатов, широко используемая технологическими компаниями на ранних этапах отбора [1].

Тип базы знаний: Гибридная документно-ориентированная база с элементами графа навыков (*skill graph*).

Формы представления знаний:

- JSON-структуры для описания задач, тестовых кейсов, решений и метаданных;
- Иерархическая модель компетенций: *язык программирования* → *домен* → *тема* → *подтема* → *задача*;
- Связи «задача → требуемый навык → минимальный уровень» для адаптивного подбора контента.

Computer Science Ontology (CSO Portal) **Назначение:** Академический проект по созданию формальной онтологии области компьютерных наук для семантической аннотации, поиска и анализа научных публикаций и образовательных ресурсов [2].

Тип базы знаний: Онтологическая база знаний на основе стандартов W3C (RDF/OWL) с поддержкой логического вывода.

Формы представления знаний:

- Формальные классы и свойства (`cso:Algorithm`, `cso:hasTimeComplexity`, `cso:requiresPrerequisite`);
- Иерархия концепций с использованием `rdfs:subClassOf`, `owl:equivalentClass`, `owl:disjointWith`;
- Связи с внешними источниками (DBpedia, Wikidata) через `owl:sameAs` для обогащения контекста.

VisuAlgo **Назначение:** Интерактивная образовательная платформа для визуализации алгоритмов и структур данных, ориентированная на улучшение понимания через анимацию и практическое исполнение [3].

Тип базы знаний: Концептуальный граф знаний (*concept graph*) с мультимодальными компонентами (текст, код, визуализация).

Формы представления знаний:

- Узлы графа представляют концепции («*Binary Heap*», «*Dijkstra's Algorithm*»);
- Рёбра кодируют отношения: `requires` (прerequisites), `visualizes` (связь с анимацией), `implemented-in` (язык реализации);
- Привязка псевдокода, реализаций на популярных языках и пошаговых анимаций к узлам графа.

Анализ структур баз знаний существующих решений

В таблице 1.3 приведено сравнительное описание структур баз знаний рассмотренных систем по ключевым критериям.

Сильные и слабые стороны рассмотренных решений:

Таблица 1.3 – Сравнительный анализ структур баз знаний аналогов

Критерий	HackerRank	CSO Portal	VisuAlgo
Виды знаний	Задачи, навыки, оценки	Концепции, отношения, аксиомы	Алгоритмы, визуализации, код
Логическая организация	Skill graph + теги + метаданные	Формальная онтология OWL 2 DL	Концептуальный граф + мультимедиа
Физическая организация	Реляционная БД + JSON-документы	RDF-триплы + SPARQL-endpoint	Файловая система + JS-граф
Механизмы вывода	Статистические рекомендации, фильтрация по профилю	Дедуктивный вывод через OWL-reasoner	Навигация по связям, интерактивное исполнение
Полнота	Высокая по задачам, средняя по теории	Высокая по академическим терминам	Высокая по визуализированным алгоритмам
Согласованность	Обеспечивается модерацией, без формальной верификации	Формальная верификация через логические ограничения	Частичная, зависит от ручной синхронизации
Расширяемость	Ограничена внутренней схемой платформы	Высокая (стандарты W3C, линковка)	Средняя (требует ручной доработки визуализаций)
Сопровождаемость	Централизованная команда, закрытый процесс	Сообщество исследователей, открытый репозиторий	Небольшая команда разработчиков

– **HackerRank:**

- *Сильные стороны:* богатая коллекция практических задач, интеграция с системами найма, адаптивная подборка на основе профиля навыков;
- *Слабые стороны:* слабая формализация теоретических знаний, отсутствие явных семантических связей между концепциями, закрытость структуры БЗ для внешнего использования.

– **CSO Portal:**

- *Сильные стороны:* строгая формальная модель, поддержка логического вывода и проверки непротиворечивости, совместимость с Semantic Web-стандартами;
- *Слабые стороны:* ориентация на академические публикации, а не на подготовку к собеседованиям, высокий порог входа для авторов контента.

– **VisuAlgo:**

- *Сильные стороны:* уникальная интеграция декларативных знаний с визуальными и процедурными компонентами, высокая педагогическая эффективность;
- *Слабые стороны:* ограниченное покрытие тем (только визуализиру-

емые алгоритмы), отсутствие механизмов адаптации под уровень пользователя.

Выводы для проектирования базы знаний проекта:

На основе проведённого анализа принято решение использовать *комбинированный подход*, заимствующий лучшие практики рассмотренных систем:

1. В качестве логической основы принять элементы онтологической модели, аналогичной CSO Portal (бинарные отношения, таксономия, ограничения целостности), для обеспечения формальной согласованности и поддержки семантического поиска;

2. Для представления контента использовать человекочитаемый формат Markdown с YAML-фронтматтером, что упростит наполнение и сопровождение по аналогии с практиками HackerRank;

3. Реализовать механизмы визуальной навигации и привязки примеров кода к концепциям, вдохновлённые подходом VisuAlgo, для повышения педагогической эффективности материалов.

Такой гибридный подход позволит совместить формальную строгость академических онтологий с практической удобностью коммерческих платформ и наглядностью образовательных ресурсов, что полностью соответствует целям разрабатываемого интеллектуального помощника по подготовке к техническим собеседованиям.

1.4 Анализ требований к базе знаний

Функциональные требования

На основании анализа типов задач и сценариев использования сформулированы следующие функциональные требования для предметной области «Фундаментальные основы Computer Science»:

1. База знаний должна обеспечивать диагностику уровня владения алгоритмическими компетенциями путём сопоставления ответов пользователя с карточками задач (PROBLEM) и вопросами (QUESTION) с использованием связей типа *evaluates* и матриц компетенций (MATRIX).

2. База знаний должна обеспечивать построение индивидуальных учебных траекторий на основе обхода графа знаний, где рёбра *prerequisite_for* и *depends_on* определяют допустимую последовательность изучения концепций (например, Массив → Динамическое программирование).

3. База знаний должна обеспечивать семантический поиск и фильтрацию единиц знаний по метаданным (домен *cs*, категория *concept/problem/pattern*, уровень сложности, роль, этап интервью, теги) с поддержкой AND-логики и ранжирования по релевантности.

4. База знаний должна обеспечивать предоставление объяснений

концепций с адаптивной глубиной изложения через извлечение данных из карточек типов **CONCEPT** и **CHEATSHEET** с учётом уровня пользователя (**junior/middle/senior**).

5. База знаний должна обеспечивать кросс-доменную интеграцию знаний путём поддержки связей между предметными областями (**cs/**, **arch/**, **process/**) и формирования единого реестра идентификаторов (**GLB-CROSS**), например, связь алгоритма **Consistent Hashing** с задачей системного дизайна **URL Shortener**.

Нефункциональные требования

Для обеспечения качества и долгосрочной жизнеспособности системы необходимо соблюдение следующих требований:

1. *Полнота покрытия.* Объём базы знаний по ПО «Фундаментальные основы CS» должен составлять не менее 200 единиц знаний, включая концепции, задачи, паттерны, вопросы, шпаргалки и типичные ошибки, согласно установленным минимумам ТЗ.

2. *Согласованность и непротиворечивость.* Терминология должна строго соответствовать глобальному глоссарию. Дублирование идентификаторов, циклические зависимости в графе предпосылок и противоречивые оценки сложности (**Big O**) в связанных документах не допускаются.

3. *Точность и актуальность.* Каждая единица знаний должна подтверждаться минимум одним авторитетным источником [4–6]. Асимптотические оценки, инварианты алгоритмов и спецификации протоколов должны быть формально корректны; дата последнего обновления (**date_updated**) для материалов по трендам не должна превышать 2 года.

4. *Расширяемость и сопровождаемость.* Структура хранения должна позволять добавление новых единиц знаний без модификации ядра системы. Использование унифицированного формата (**Markdown + YAML-фронтматтер**) необходимо для обеспечения версионирования, автоматизированной валидации и последующей интеграции с решателями задач.

5. *Производительность обработки.* Операции поиска, фильтрации и валидации перекрёстных ссылок должны выполняться в интерактивном режиме (до 2 секунд для выборки до 50 единиц) при использовании штатного **CLI**-инструмента.

Требования к видам знаний и формализму

База знаний должна поддерживать репрезентативный набор видов знаний, релевантных задачам подготовки к техническим собеседованиям в области **Computer Science**:

– *Декларативные знания:* факты о свойствах структур данных (**CONCEPT**), таксономии алгоритмических парадигм (**PATTERN**), описания вычислительной сложности и ограничений целостности (инварианты, предусловия/постусловия).

– *Процедурные знания:* алгоритмические схемы решения типовых

задач, паттерны декомпозиции (**Sliding Window, Two Pointers**), чек-листы анализа сложности (**CHEATSHEET**), критерии оценки корректности кода.

– *Метазнания и ограничения*: правила валидации структуры документов, матрицы соответствия навыков уровням сложности, учебные траектории (**TRAJECTORY**), правила калибровки сложности задач (**LeetCode Easy/Medium/Hard** $\leftrightarrow$ **junior/middle/senior**).

В качестве основного формализма представления знаний выбран полуструктурированный документно-ориентированный подход на базе Markdown с YAML-фронтматтером [7]. Данный выбор обоснован следующими факторами:

1. *Выразительная мощность*: YAML обеспечивает строгую типизацию метаданных (идентификаторы формата **CS-CATEGORY-NN**, домены, теги, связи), а Markdown позволяет хранить развёрнутые объяснения, ASCII-диаграммы структур данных и примеры кода на псевдокоде/Python, что критично для алгоритмической предметной области.

2. *Удобство обработки*: Формат парсится стандартными библиотеками (**pyyaml**), что позволяет реализовать автоматизированную валидацию, генерацию реестров и семантический поиск без развёртывания тяжёлых СУБД.

3. *Интеграционный потенциал*: Документная структура совместима с конвейерами RAG (векторизация, чанкинг), легко конвертируется в RDF/OWL при необходимости перехода к онтологическому формализму, и полностью поддерживается системами контроля версий (Git) для совместной работы трёх инженеров знаний.[6, 7]

Не допускается смешение несовместимых формализмов без явной схемы согласования. Все связи между документами должны быть явно зафиксированы в полях **related** и **prerequisites** с указанием типа связи в комментариях (например, **# depends_on**, **# applied_in**).

1.5 Вывод по разделу анализа

На основании проведённого исследования можно сформулировать следующие итоги, определяющие направление проектирования базы знаний для предметной области «Фундаментальные основы Computer Science»:

1. Предметная область требует представления знаний, ориентированного не только на декларативные описания алгоритмов, но и на процедурные паттерны решения задач (**PATTERN**), анализ типичных ошибок (**PITFALL**) и адаптацию глубины изложения под уровень кандидата.

2. Существующие образовательные платформы демонстрируют ограничения в формализации семантических связей между концепциями, поддержке персонализированной диагностики алгоритмических навыков и глубокой проработке компромиссов (time vs. space complexity), что обосно-

вызывает необходимость разработки специализированной базы знаний.

3. База знаний должна решать задачи диагностики компетенций, планирования учебных траекторий, семантического поиска и генерации объяснений с варьированием детализации, что требует внедрения графовой модели отношений (`prerequisite_for`, `depends_on`, `applied_in`, `warns_about`).

4. Функциональные требования диктуют необходимость поддержки фильтрации по метаданным (домен, категория, сложность, роль, этап интервью), обхода графа зависимостей и кросс-доменной интеграции с предметными областями ARCH и PROCESS.

5. Нефункциональные требования фиксируют минимальный объём (≥ 200 единиц для ПО «CS»), строгую согласованность терминологии, обязательную верификацию источников [4, 8] и использование полуструктурированного формата Markdown+YAML для обеспечения машиночитаемости, расширяемости и совместной работы.

6. В проектировочном разделе будет обоснован выбор документно-графовой архитектуры БЗ, реализована схема валидации метаданных через CLI-инструмент, определены форматы входных/выходных данных для модулей диагностики и рекомендаций, и разработаны диаграммы компонентов системы, соответствующие сформулированным требованиям.

2 ПРОЕКТИРОВАНИЕ БАЗЫ ЗНАНИЙ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Общая архитектура интеллектуальной системы и место базы знаний

Архитектурная организация интеллектуальной системы «TechInterview» реализована по модульно-компонентному принципу, где база знаний фундаментальных основ Computer Science выполняет функцию центрального семантического ядра, обеспечивающего хранение, структурирование и предоставление доступа к предметно-ориентированным материалам. Декомпозиция системы знаний выполнена на четыре предметные области (ПО) уровня 1 и интеграционный уровень 0, что позволяет обеспечить масштабируемость, изоляцию ответственности и эффективную кросс-доменную навигацию.

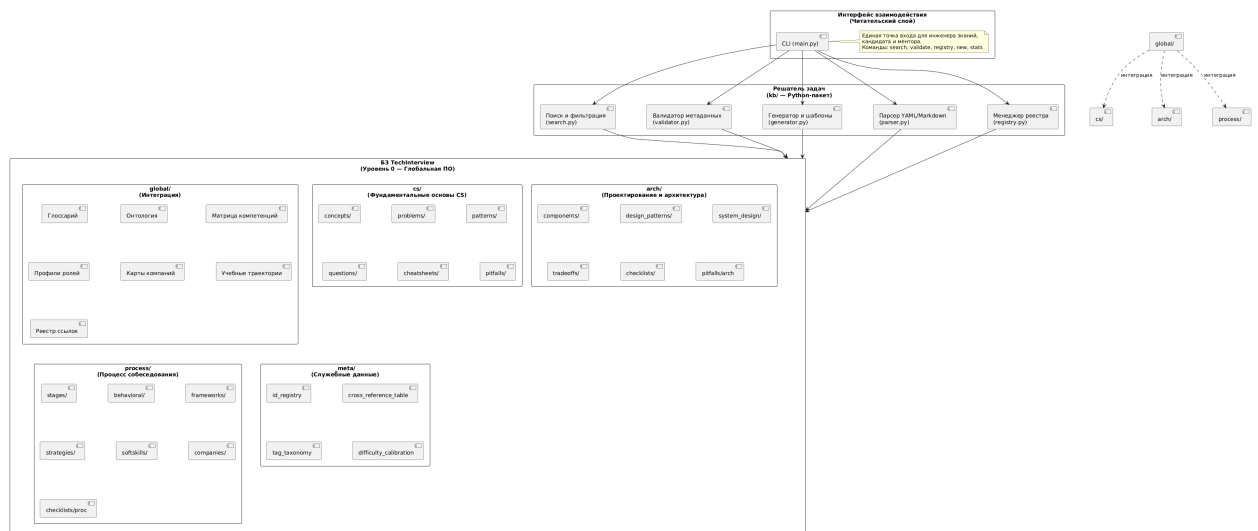


Рисунок 2.1 – Уровневая организация базы знаний (Уровень 0 — глобальная ПО). Пунктирные линии обозначают интеграционные связи между глобальным слоем и доменными областями.

Согласно архитектурной схеме, представленной на рисунке 2.1, система включает следующие функциональные компоненты:

– **База знаний (Knowledge Base)** — распределённое файловое хранилище, структурированное по доменам (cs/, arch/, process/, global/). Каждый домен инкапсулирует специфичные типы единиц знаний, соответствующие таксономии и правила валидации. Для предметной области «Фундаментальные основы CS» это включает карточки концепций (CONCEPT), алгоритмические задачи (PROBLEM), паттерны решения (PATTERN), вопросы собеседования (QUESTION), шпаргалки (CHEATSHEET) и описания типичных ошибок (PITFALL).

– **Модуль логического вывода (Solver/Reasoner)** — программный слой на языке Python, реализующий алгоритмы семантического поиска с учётом релевантности, валидации метаданных по заданным схемам, топологической сортировки графа зависимостей (`prerequisite_for`), формирования адаптивных учебных траекторий на основе выявленных пробелов и диагностики уровня владения алгоритмическими компетенциями. Модуль поддерживает как детерминированные правила вывода, так и эвристические стратегии ранжирования результатов.

– **Слой взаимодействия (Interface Layer)** — многоканальный интерфейс, включающий консольный интерфейс (CLI) для инженеров знаний и веб-интерфейс (MVP) для конечных пользователей. Консольный интерфейс предоставляет команды для управления полным жизненным циклом единиц знаний: создание по шаблону, редактирование контента, валидация метаданных, семантический поиск с комбинированными фильтрами, генерация реестров и статистических отчётов. Веб-интерфейс обеспечивает визуализацию карточек знаний, навигацию по графу связей и интерактивные режимы самопроверки.

– **Инфраструктурные сервисы** — система контроля версий Git для отслеживания изменений на уровне отдельных файлов и аудита истории редактирования, внешние источники знаний (академические учебники, документация технологий, инженерные блоги) для верификации информации, а также механизмы автоматизированного тестирования для контроля качества валидаторов и поисковых алгоритмов.

База знаний функционирует в **режиме чтения** для пользовательских сценариев: модули решателя, интерфейсы и внешние интеграции обращаются к ней исключительно для извлечения данных. Внесение изменений осуществляется исключительно инженерами знаний через CLI-инструмент с обязательной процедурой коммит-валидации, включающей проверку формата метаданных, уникальности идентификаторов и целостности перекрёстных ссылок. Такой подход обеспечивает контроль целостности, воспроизводимость изменений и возможность отката к предыдущим версиям при обнаружении ошибок [7].

Для предметной области CS дополнительно реализованы специализированные механизмы:

– *Калибровка сложности*: автоматическая привязка единиц знаний к уровням `junior/middle/senior` на основе метрик (асимптотическая сложность алгоритма, глубина требуемых прerreквизитов, частота упоминания в собеседованиях).

– *Граф алгоритмических зависимостей*: ориентированный ациклический граф, где вершины — концепции и задачи, а рёбра типа `prerequisite_for` определяют допустимые последовательности изучения (например, Массив → Двоичный поиск → Динамическое программирование).

– *Матрица компетенций*: таблица соответствия между единицами знаний и проверяемыми навыками (понимание асимптотического анализа, умение реализовывать обход графа, способность оптимизировать решение), используемая для диагностики и персонализации траекторий.

2.2 Выбор формализма и логической основы

Описание выбранного формализма

В качестве базового формализма представления знаний принят документно-ориентированный полуструктурированный подход на основе **Markdown с YAML-фронтматтером** [7]. Данный выбор обусловлен комплексом технических и методологических факторов:

1. *Соответствие задачам предметной области*: Формализм позволяет совмещать развёрнутые текстовые описания алгоритмических концепций, примеры кода на псевдокоде/Python, ASCII-диаграммы структур данных со строго типизированными метаданными. Это критически важно для реализации задач диагностики уровня владения алгоритмами, планирования последовательностей изучения тем с учётом пререквизитов и генерации проверочных вопросов с вариативной глубиной изложения.

2. *Выразительность и гибкость*: YAML поддерживает вложенные структуры данных, списки связей (**related**, **prerequisites**), таксономическую разметку тегов и перечисления с ограниченным набором значений. В то же время Markdown обеспечивает возможность встраивания блоков кода с подсветкой синтаксиса, многоуровневых заголовков для структурирования контента, таблиц для сравнения алгоритмов и текстовых диаграмм для визуализации структур данных [6].

3. *Технологическая совместимость и простота обработки*: Формат обрабатывается стандартными библиотеками (**pyyaml**, **markdown**), не требует развёртывания специализированных СУБД, полностью совместим с системами контроля версий (Git) для совместной работы нескольких инженеров знаний и легко интегрируется в конвейеры RAG (Retrieval-Augmented Generation) для векторного поиска и семантического ранжирования.

4. *Эволюционная гибкость и обратная совместимость*: Полуструктурированная схема позволяет расширять набор метаданных без миграции существующих данных: новые атрибуты (например, **time_complexity_best**, **space_complexity_worst**) добавляются в YAML-блок без нарушения обратной совместимости с уже реализованными модулями поиска и валидации [9].

5. *Поддержка машиночитаемости и человекочитаемости*: YAML-фронтматтер обеспечивает строгую типизацию, необходимую для автоматизированной обработки, в то время как Markdown-тело документа остаётся удобным для чтения и редактирования человеком, что снижает порог входа

для авторов контента и упрощает процесс рецензирования.

На рисунке 2.2 представлена структура типовой единицы знаний на примере карточки концепции «Хеш-таблица», иллюстрирующая разделение метаданных (фронтматтер) и содержательной части, а также взаимосвязь атрибутов, ссылок на связанные документы и уровней детализации для разных уровней подготовки.

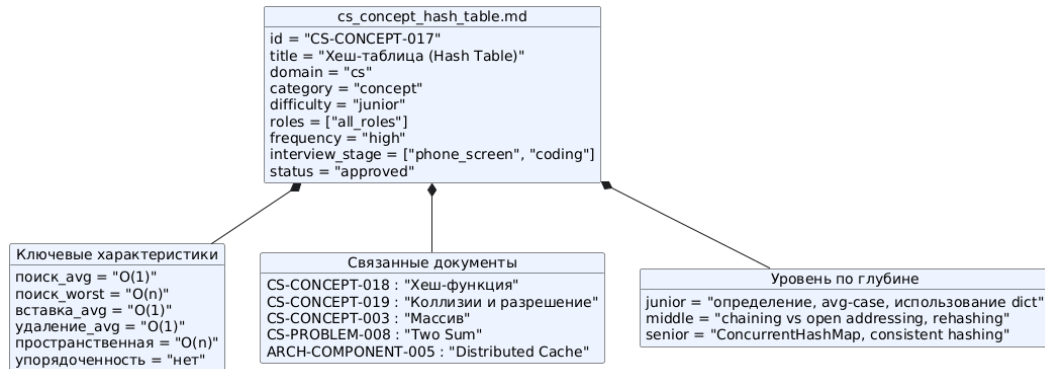


Рисунок 2.2 – Структура единицы знаний: взаимосвязь метаданных YAML, атрибутов, ссылок на связанные документы и уровней детализации.

Синтаксис и семантика

Синтаксическая и семантическая организация базы знаний формализована следующим образом:

- **Факты и сущности** описываются полями YAML-фронтматтера (id, title, domain, category, tags, difficulty, roles, interview_stage, frequency, related, prerequisites, sources, author, version, date_created, date_updated, status). Каждое поле имеет строгую типизацию: строковые значения, перечисления (enum) или массивы. Наличие обязательных полей (id, title, domain, category, difficulty) формирует минимальный контракт для каждой единицы знаний, гарантирующий её корректную обработку модулями системы.

- **Отношения** фиксируются в массивах related и prerequisites, где элементы представляют ссылки на идентификаторы других единиц. Семантика связи определяется типом отношения, указываемым в комментарии: prerequisite_for (предпосылка для изучения), depends_on (зависимость реализации), evaluates (проверка компетенции), contrasts_with (сравнение альтернатив), applied_in (применение в задаче), warns_about (предупреждение об ошибке).

- **Ограничения целостности** реализованы на уровне модуля валидации: проверяется соответствие формата ID регулярному выражению $[A-Z]+[A-Z]+\{d\{3\}\$$, допустимость значений перечислений (домены, категории, уровни сложности), уникальность идентификаторов в пределах всей БЗ, валидность всех перекрёстных ссылок (физическое существование фай-

лов с указанными ID) и наличие хотя бы одного авторитетного источника в поле `sources`.

- **Правила генерации идентификаторов:** ID формируются по шаблону [ДОМЕН] - [КАТЕГОРИЯ] - [НН] (например, CS-CONCEPT-017, CS-PROBLEM-008), где домен (CS, ARCH, PROC, GLB) и категория (CONCEPT, PROBLEM, PATTERN и др.) кодируются заглавными латинскими буквами, а порядковый номер — трёхзначным числом с ведущими нулями для обеспечения лексикографической сортируемости.

- **Таксономия тегов:** теги выбираются из утверждённой таксономии (`_meta/tag_taxonomy.md`), включающей группы: техническая тема (`arrays, hash_tables, dynamic_programming`), приоритет (`must_know, core, advanced`), частота на собеседованиях (`frequently_asked, occasionally_asked`) и роль (`backend, frontend, all_roles`). Произвольные теги не допускаются для обеспечения согласованности поиска.

Использование смешанных или несовместимых формализмов запрещено. При необходимости интеграции с онтологическими системами в будущем будет разработан модуль экспорта, транслирующий YAML-метаданные в стандарт RDF/OWL с сохранением семантики связей и ограничений.

2.3 Концептуальная модель предметной области

Концептуальная модель предметной области «Фундаментальные основы Computer Science» базируется на абстрактной сущности **Единица Знаний (KnowledgeUnit)**, от которой наследуются специализированные типы, соответствующие шести категориям единиц знаний, определённым в техническом задании. Ключевые атрибуты сущностей включают: уникальный идентификатор (`id`), заголовок (`title`), принадлежность к домену и категории, уровень сложности (`difficulty`), целевые роли (`roles`), этапы интервью (`interview_stage`), набор тегов (`tags`), статус валидации (`status`), список источников (`sources`) и метаданные авторства и версионирования.

Отношения между сущностями формируют ориентированный ациклический граф знаний со следующими типами связей, специфичными для алгоритмической предметной области:

- *Иерархические:* `is_part_of` (концепция является частью более общей темы), `extends` (расширение базовой концепции, например, BST → AVL Tree).

- *Семантические:* `relates_to` (тематическая связь, например, Hash Table ↔ Hash Function), `contrasts_with` (сравнение альтернатив, например, Array vs Linked List), `alternative_to` (альтернативный подход к решению).

- *Учебные зависимости:* `prerequisite_for` (необходимое предвари-

тельное знание, например, Рекурсия → Динамическое программирование), `depends_on` (зависимость реализации, например, Dijkstra → Priority Queue).

– *Оценочные*: `evaluates` (связь задачи/вопроса с проверяемой компетенцией), `applied_in` (применение концепции в конкретной задаче, например, Two Pointers → Two Sum), `warns_about` (предупреждение о типичной ошибке, связанной с концепцией).

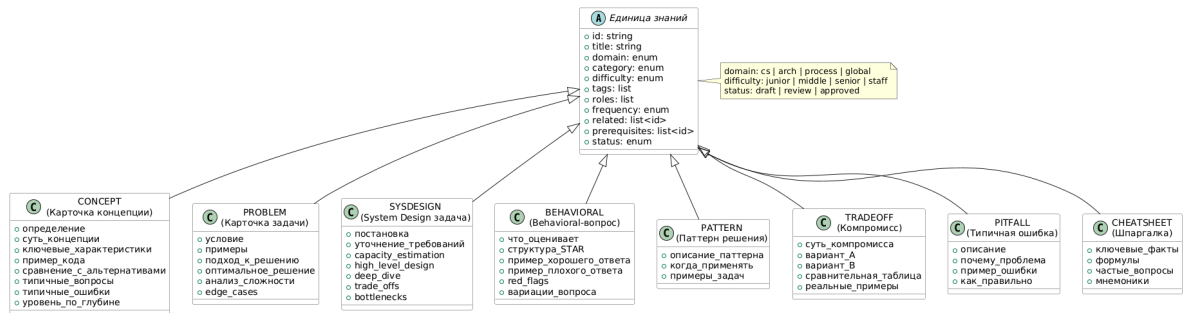


Рисунок 2.3 – Концептуальная модель: иерархия классов единиц знаний и их специфические атрибуты для предметной области Computer Science.

Структура концептов напрямую обеспечивает решение ключевых задач системы подготовки к собеседованиям:

– Наличие связей типа `prerequisite_for` позволяет модулю планирования выстраивать корректные последовательности изучения тем, исключая ситуации, когда кандидат пытается освоить сложную концепцию без необходимой базовой подготовки.

– Отношения `evaluates` обеспечивают механизм диагностики пробелов через сопоставление ответов пользователя с матрицей компетенций, позволяя системе рекомендовать конкретные карточки для повторения.

– Кластеризация единиц типов `PITFALL` и `CHEATSHEET` формирует базу для генерации персонализированных предупреждений и шпаргалок, адаптированных под выявленные слабые места кандидата.

– Типы `QUESTION` и `PROBLEM` с эталонными ответами, разборами сложности и примерами кода поддерживают режим интерактивной самопроверки с автоматической оценкой корректности решения.

– Связи `applied_in` и `exemplifies` позволяют продемонстрировать практическое применение теоретических концепций в контексте реальных задач собеседований, повышая педагогическую эффективность материалов.

Для предметной области CS дополнительно введены специализированные атрибуты:

– `time_complexity_avg`, `time_complexity_worst`, `space_complexity` — для карточек алгоритмов и структур данных.

– `input_constraints`, `edge_cases` — для карточек задач.

- `pattern_name`, `recognition_hints` — для карточек паттернов решения.
- `common_mistakes`, `how_to_avoid` — для карточек типичных ошибок.

2.4 Логическая и физическая структура базы знаний

Логическая структура

Логическая организация БЗ для предметной области «Фундаментальные основы Computer Science» реализована в виде иерархической структуры каталогов, отражающей тематическую декомпозицию домена:

- `cs/concepts/` — **Карточки концепций:** Содержит теоретические описания фундаментальных концепций CS: структуры данных (массивы, связные списки, деревья, графы, хеш-таблицы, кучи), алгоритмы (сортировки, поиски, обходы графов, динамическое программирование), принципы (инкапсуляция, абстракция, асимптотический анализ). Каждая карточка включает определение, ключевые характеристики, визуальное представление, пример кода, сценарии применения, сравнение с альтернативами, типичные вопросы на собеседовании и уровни детализации для `junior/middle/senior`.

- `cs/problems/` — **Алгоритмические задачи:** Включает задачи с условиями, примерами ввода-вывода, ограничениями, уточняющими вопросами, пошаговым разбором решения (от наивного подхода к оптимальному), анализом сложности, обработкой граничных случаев и связями с концепциями и паттернами. Задачи классифицированы по темам (`arrays/`, `strings/`, `trees/`, `graphs/`, `dynamic_programming/`) и уровню сложности.

- `cs/patterns/` — **Паттерны решения задач:** Содержит обобщённые стратегии применения знаний в контексте интервью: `Sliding Window`, `Two Pointers`, `Fast & Slow Pointers`, `Merge Intervals`, `Cyclic Sort`, `Top K Elements`, `Modified Binary Search`. Для каждого паттерна указаны признаки распознавания, шаблон реализации, примеры задач и связи с концепциями.

- `cs/questions/` — **Вопросы собеседования:** Включает теоретические вопросы с эталонными ответами, критериями оценки и ссылками на релевантные концепции. Вопросы сгруппированы по темам и уровням сложности.

- `cs/cheatsheets/` — **Шпаргалки:** Компактные справочные материалы с ключевыми формулами, определениями, шаблонами кода и мнемоническими правилами для быстрого повторения перед собеседованием.

- `cs/pitfalls/` — **Типичные ошибки:** Описания распространённых заблуждений и ошибок кандидатов (например, «Путаница между средним и худшим случаем сложности», «Некорректная оценка пространственной сложности рекурсии») с пояснениями, как их избежать.

Границы модулей определяются строгим соответствием кода домена и категории в идентификаторах. Модули изолированы по зоне ответственности, но связаны через единое пространство идентификаторов и механизмы кросс-доменных ссылок с другими предметными областями (например, связь алгоритма `Consistent Hashing` с задачей системного дизайна `URL Shortener` из домена `arch/`).

Физическая структура и форматы хранения

Физически база знаний представляет собой распределённое файловое хранилище в формате Markdown (`.md`) с кодировкой UTF-8. Каждая единица знаний хранится в отдельном файле, имя которого следует шаблону `{домен}_{категория}_{slug}.md`, где `slug` — краткое название на латинице в `snake_case` (например, `cs_concept_hash_table.md`, `cs_problem_two_sum.md`).

Первичным ключом выступает поле `id` в YAML-фронтматтере. Для обеспечения нормализации применяется принцип единственного источника истины (SSOT): каждый факт описывается ровно один раз, повторное использование достигается через механизм перекрёстных ссылок. Файлы организованы в иерархическую структуру каталогов, отражающую тематическую группировку:

```
|-- concepts/
|   |-- data_structures/
|   |   |-- cs_concept_array.md
|   |   |-- cs_concept_linked_list.md
|   |   |-- cs_concept_hash_table.md
|   |   \-- ...
|   |-- algorithms/
|   |   |-- cs_concept_binary_search.md
|   |   |-- cs_concept_quicksort.md
|   |   |-- cs_concept_dynamic_programming.md
|   |   \-- ...
|   \-- ...
|-- problems/
|   |-- arrays/
|   |   |-- cs_problem_two_sum.md
|   |   \-- ...
|   |-- strings/
|   |   \-- ...
|-- patterns/
|   |-- cs_pattern_sliding_window.md
|   |-- cs_pattern_two_pointers.md
|   \-- ...
```

```
|-- questions/  
|-- cheatsheets/
```

Такая организация обеспечивает:

- **Версионированность:** Система контроля версий Git отслеживает изменения на уровне отдельных файлов, позволяя просматривать историю редактирования, сравнивать версии и откатываться при необходимости.
- **Параллельную разработку:** Несколько инженеров знаний могут одновременно редактировать разные файлы без конфликтов, так как каждая единица знаний изолирована в своём файле.
- **Эффективный поиск:** Индексация метаданных позволяет выполнять фильтрацию по домену, категории, сложности, тегам и другим атрибутам без загрузки полного содержимого документов, что обеспечивает интерактивное время отклика.
- **Горизонтальную масштабируемость:** Добавление новых единиц знаний не требует модификации схемы данных или остановки системы — достаточно создать новый файл в соответствующем каталоге и обновить реестр идентификаторов [9].
- **Простоту резервного копирования и миграции:** Файловая структура легко копируется, архивируется и переносится между средами без необходимости экспорта/импорта в специализированные форматы.

2.5 Требования к программному прототипу (MVP)

Назначение и сценарии использования

Программный прототип (MVP) представляет собой веб-интерфейс для интерактивной демонстрации возможностей базы знаний предметной области CS. Он выполняет роль читательского слоя, обеспечивая наглядную визуализацию семантической связности материалов, полноты покрытия ключевых тем алгоритмов и структур данных, а также корректности реализации механизмов поиска и навигации [7].

Основные сценарии использования отражены на диаграмме вариантов использования (Use Case) на рисунке 2.4. Система поддерживает три роли пользователей, для каждой из которых определён набор доступных действий:

- **Кандидат** (уровни Junior/Middle/Senior) — выполняет поиск и фильтрацию материалов по темам, сложности и тегам; просматривает карточки концепций с адаптивной глубиной изложения; навигирует по семантическим связям между задачами, паттернами и теоретическими концепциями; проходит режимы самопроверки с скрытыми эталонными ответами; получает персонализированные рекомендации по устранению выявленных пробелов.

– **Инженер знаний** — осуществляет валидацию метаданных новых и редактируемых единиц знаний; управляет статусами единиц (**draft/review/approved**); контролирует целостность графа перекрёстных ссылок; обновляет реестр идентификаторов; генерирует статистические отчёты о покрытии тем и распределении единиц по уровням сложности.

– **Ментор / Технический интервьюер** — использует базу для формирования вопросов и задач, соответствующих целевому уровню кандидата; навигирует по связанным темам для углубления дискуссии; оценивает компетенции кандидатов на основе матрицы навыков; генерирует индивидуальные планы подготовки для подопечных.

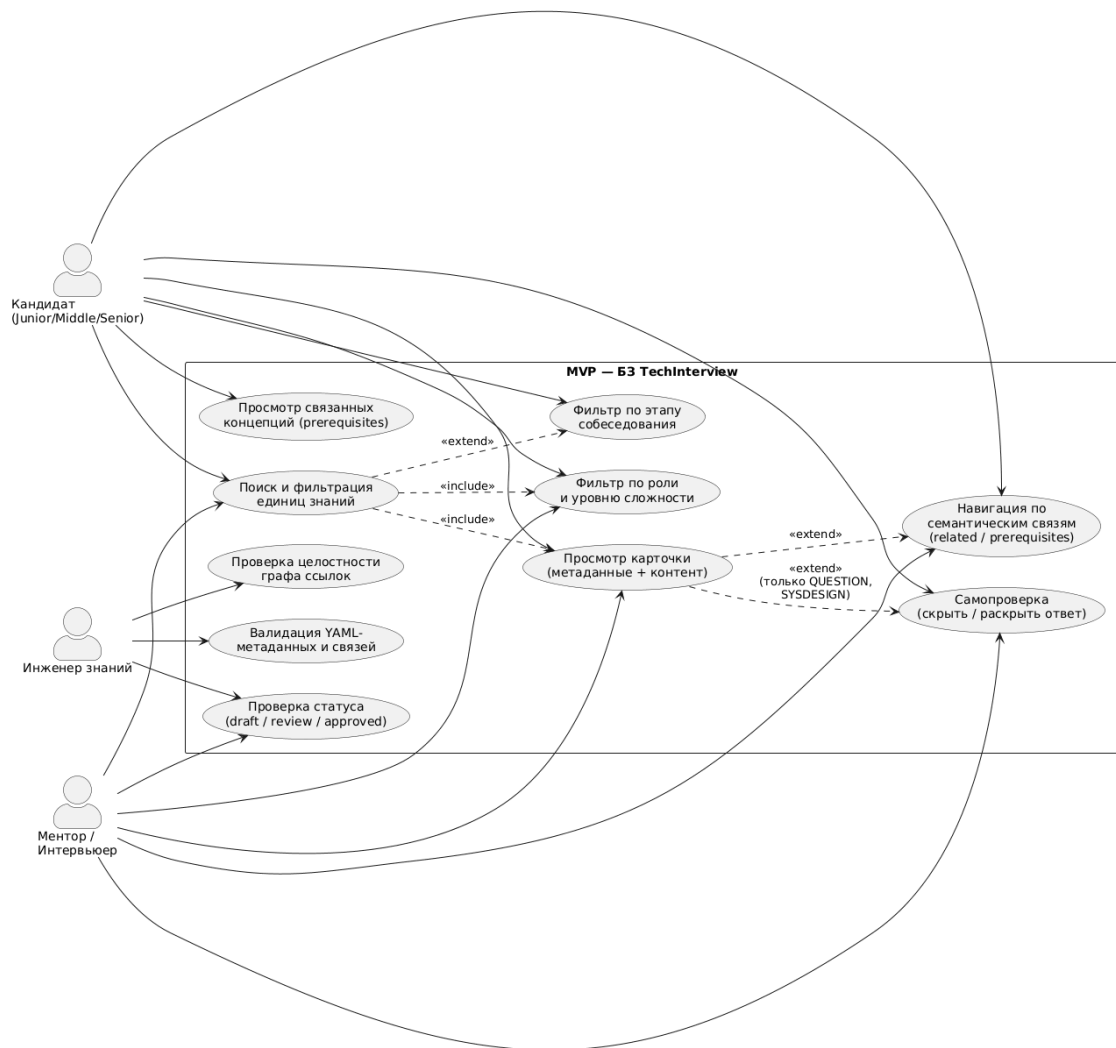


Рисунок 2.4 – Диаграмма вариантов использования (Use Case) MVP-приложения, отражающая роли и основные сценарии взаимодействия для предметной области CS.

Функциональные требования

На основании сценариев использования к прототипу предъявляются следующие функциональные требования:

1. Прототип должен обеспечивать корректный рендеринг единиц знаний с отображением YAML-метаданных (идентификатор, домен, категория, сложность, теги, статус) и структурированного Markdown-контента (заголовки, списки, таблицы, блоки кода с подсветкой синтаксиса для языков Python/псевдокод, ASCII-диаграммы структур данных).

2. Прототип должен обеспечивать комбинированный поиск и фильтрацию по метаданным: домену (`cs`), категории (`concept/problem/pattern` и др.), уровню сложности (`junior/middle/senior`), целевой роли (`backend/frontend/all_roles`), этапу собеседования (`coding/phone_screen`) и тегам (с поддержкой логической операции AND при указании нескольких тегов).

3. Прототип должен обеспечивать полнотекстовый поиск по заголовку, идентификатору, тегам и содержимому документа с регистронезависимым сопоставлением и ранжированием результатов по релевантности (совпадение в заголовке имеет больший вес, чем в теле документа).

4. Прототип должен обеспечивать интерактивную навигацию по семантическим связям между единицами знаний через кликабельные ссылки на поля `related` и `prerequisites`, с визуальной индикацией типа связи (например, иконками для `prerequisite_for`, `applied_in`, `warns_about`).

5. Прототип должен обеспечивать базовый режим самопроверки для карточек типов `QUESTION` и `PROBLEM`: скрытие эталонного ответа и разбора решения до момента взаимодействия пользователя с элементом управления «Показать ответ», последующее раскрытие ответа с критериями оценки и пояснениями типичных ошибок.

6. Прототип должен обеспечивать визуальную индикацию статуса единицы знаний (цветовая маркировка для `draft/review/approved`) и предупреждать пользователя при попытке обращения к материалам со статусом черновика, предлагая альтернативные утверждённые единицы.

7. Прототип должен обеспечивать адаптивное отображение контента в зависимости от уровня подготовки пользователя: для `junior` — базовые определения и примеры, для `middle` — дополнительные детали реализации и анализ сложности, для `senior` — углублённые сравнения альтернатив и обсуждение компромиссов.

Архитектурные решения

Архитектура MVP реализована по модели одностраничного клиентского приложения (SPA) с локальным сервером раздачи статических файлов.

Отказ от полноценного серверного бэкенда обоснован тем, что база знаний уже обладает самодостаточной структурой: каждый `.md` файл является автономной единицей данных со встроенными метаданными, что позволяет выполнять поиск, фильтрацию и рендеринг непосредственно на стороне клиента [6].

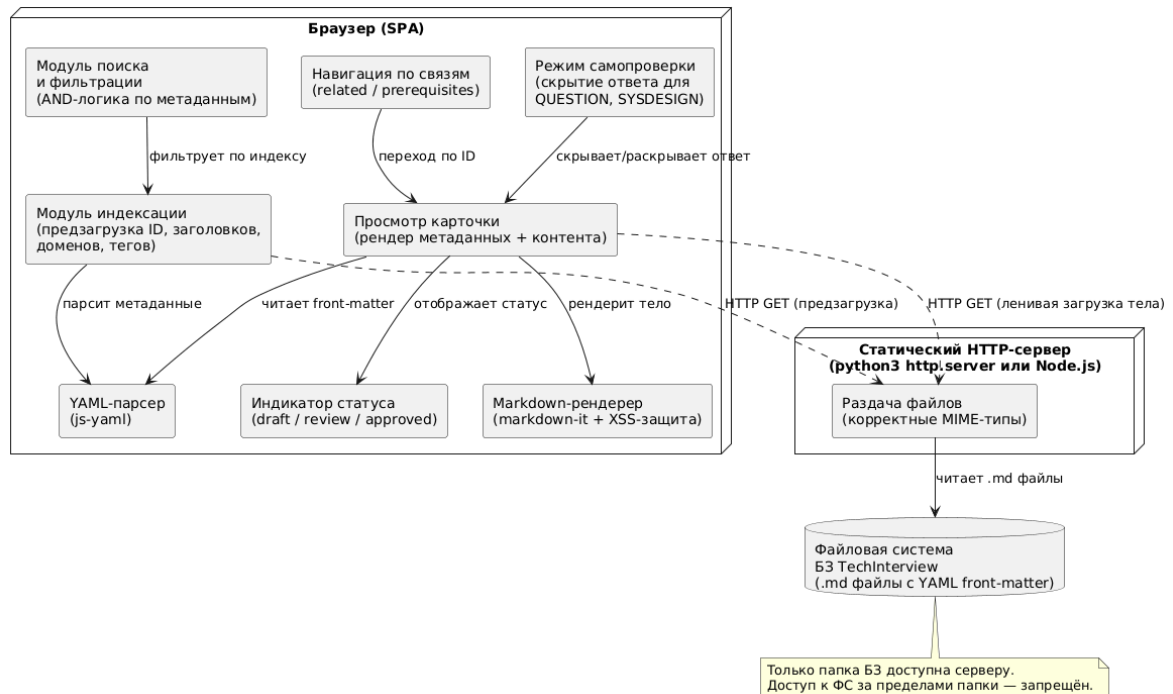


Рисунок 2.5 – Архитектурная схема взаимодействия компонентов MVP: клиентская индексация метаданных, парсинг фронтматтера и ленивая загрузка содержимого документов.

Как показано на рисунке 2.5, архитектура включает следующие компоненты:

- **Клиентский слой (Browser SPA):**
 - *Модуль поиска и фильтрации:* работает с предзагруженным индексом метаданных (файл `_meta/id_registry.md`), позволяя выполнять фильтрацию по атрибутам без загрузки полного содержимого документов.
 - *YAML-парсер:* извлекает метаданные из фронтматтера для отображения в интерфейсе и использования в логике фильтрации.
 - *Markdown-рендерер:* преобразует тело документа в HTML с поддержкой подсветки синтаксиса кода, таблиц и многоуровневых заголовков.
 - *Модуль навигации:* обрабатывает клики по семантическим ссылкам, загружая связанные документы «на лету».
- **Серверный слой:** Статический HTTP-сервер (например, `python3 -m http.server`), выполняющий исключительно раздачу файлов с корректными MIME-типами (`text/markdown` для `.md`, `text/yaml` для фронтматтера).

– **Файловая система:** Хранилище .md файлов, доступное в пределах корневой директории проекта, с иерархической структурой каталогов, отражающей логическую организацию знаний.

Такая архитектура обеспечивает высокую производительность (мгновенная фильтрация на стороне клиента за счёт предварительно построенного индекса) и минимизацию сетевого трафика за счёт ленивой загрузки тела документа только при переходе к конкретной карточке. Кроме того, отсутствие серверной логики упрощает развёртывание прототипа и снижает требования к инфраструктуре.

2.6 Обеспечение согласованности, расширяемости и сопровождаемости

Механизмы контроля согласованности:

1. *Строгая типизация метаданных:* YAML-фронтматтер валидируется на соответствие допустимым перечислениям (домены, категории, уровни сложности, статусы), что исключает появление произвольных значений в таксономии и обеспечивает единообразие классификации. Валидация выполняется как на этапе коммита (через CLI-инструмент), так и при загрузке в веб-интерфейс.

2. *Валидация формата идентификаторов:* Реализована проверка соответствия ID регулярному выражению $[A-Z] + [A-Z] + \backslash d\{3\}\$,$ гарантирующая уникальность и предсказуемую структуру всех перекрёстных ссылок. Дублирование идентификаторов блокируется на уровне валидатора с указанием конфликтующих файлов.

3. *Целостность графа зависимостей:* Модуль валидации сканирует поля **related** и **prerequisites**, проверяя физическое существование файлов с указанными идентификаторами. При обнаружении «висячих» ссылок генерируется предупреждение с рекомендацией исправить ссылку или создать отсутствующий документ.

4. *Управление жизненным циклом:* Внедрён механизм статусов (**draft**, **review**, **approved**), предотвращающий попадание непроверенной информации в обучающий контур для конечных пользователей. Переход в статус **approved** возможен только после рецензирования другим участником команды, что обеспечивает двухуровневый контроль качества.

5. *Контроль терминологической согласованности:* Все технические термины нормализуются согласно глобальному глоссарию (**global/glossary/**), что исключает синонимическую избыточность и обеспечивает единообразие изложения в разных карточках. При первом упоминании термина в документе даётся русское пояснение с английским оригиналом в скобках.

Расширяемость и модульность:

1. *Доменная изоляция*: Знания разделены на три независимые предметные области и один интеграционный слой, что позволяет параллельно развивать разные аспекты системы без взаимного влияния. Инженер знаний №1 может добавлять новые карточки в `cs/`, не затрагивая логику доменов `arch/` и `process/`.

2. *Документно-ориентированный подход*: Каждая единица хранится в отдельном файле. Добавление нового понятия сводится к созданию одного файла без необходимости миграции схемы данных, остановки системы или перезагрузки индексов. Это позволяет инкрементально пополнять базу знаний в процессе разработки курсового проекта.

3. *Динамический реестр*: Глобальный индекс связей (`_meta/id_registry.md`) генерируется автоматически по запросу через команду `python3 main.py registry`, что позволяет графу знаний эволюционировать динамически по мере пополнения базы. Реестр включает статистику по доменам, категориям и статусам, а также предупреждения о дубликатах.

4. *Расширяемость схемы метаданных*: Полуструктурированный формат YAML позволяет добавлять новые поля в фронтматтер без нарушения обратной совместимости. Например, при необходимости поддержки новых атрибутов (например, `leetcode_difficulty`, `company_frequency`) достаточно обновить валидатор и шаблоны, не затрагивая существующие файлы.

Сопровождаемость и документирование:

1. *Шаблоны единиц знаний*: Для каждой категории (`concept`, `problem`, `pattern` и др.) предусмотрен стандартизированный шаблон с обязательными секциями, что упрощает создание нового контента и обеспечивает единообразие структуры документов.

2. *CLI-инструменты поддержки*: Набор команд (`new`, `validate`, `registry`, `stats`, `search`) автоматизирует рутинные операции по управлению базой знаний, снижая вероятность человеческих ошибок и ускоряя процесс разработки.

3. *Тестирование и валидация*: Реализован набор из 177 автоматизированных тестов (через `pytest`), покрывающих функциональность парсеров, валидаторов, поисковых алгоритмов и генераторов. Запуск тестов перед каждым коммитом обеспечивает стабильность системы.

4. *Документация для авторов*: Файлы `CONTRIBUTING.md` и `TERMINAL_GUIDE.md` содержат подробные инструкции по добавлению и редактированию единиц знаний, оформлению метаданных, использованию тегов и работе с перекрёстными ссылками.

2.7 Вывод по разделу проектирования

В ходе проектирования базы знаний интеллектуальной системы `TechInterview` для предметной области «Фундаментальные основы

Computer Science» были приняты следующие ключевые архитектурные и технологические решения:

1. Выбор формализма представления: Комбинация Markdown и YAML выбрана как оптимальный баланс между читаемостью для человека и машиночитаемостью, что удовлетворяет требованию к поддержке как декларативных (теоретические концепции), так и процедурных (алгоритмы, паттерны решения) знаний [6, 7]. Полуструктурированный подход обеспечивает гибкость эволюции схемы метаданных без нарушения обратной совместимости.

2. Многоуровневая архитектура: Декомпозиция на доменные области и выделение глобального интеграционного слоя позволяет реализовать требования к кросс-доменным связям (например, связь алгоритма **Consistent Hashing** с задачей системного дизайна) без дублирования информации. Модульная структура соответствует принципу единственной ответственности, упрощая тестирование и сопровождение отдельных компонентов [9, 10].

3. Инструментальная поддержка разработки: Внедрение CLI-инструмента с функциями валидации, поиска, генерации шаблонов и управления реестром обеспечивает контроль качества на этапе ввода данных, снижает вероятность человеческих ошибок и ускоряет процесс пополнения базы знаний. Автоматизированные тесты (177 тестовых случаев) гарантируют стабильность ключевых модулей системы [9].

4. Подготовка к интеграции с решателем: Чёткая структура идентификаторов (CS-CATEGORY-NN), граф семантических связей (**prerequisite_for**, **applied_in**, **evaluates**) и матрица компетенций закладывают фундамент для разработки модуля логического вывода, который сможет использовать эти связи для построения адаптивных учебных траекторий, диагностики пробелов и генерации персонализированных рекомендаций [4, 8].

5. Горизонтальная масштабируемость: Файловая структура позволяет базе знаний расти линейно, поддерживая плановый объём более 200 единиц знаний для домена CS без деградации производительности операций поиска и фильтрации. Индексация метаданных и ленивая загрузка контента обеспечивают интерактивное время отклика даже при увеличении размера базы [6].

6. Многоуровневый контроль качества: Система валидации метаданных (формат ID, допустимые значения перечислений, целостность ссылок), механизм рецензирования (двухуровневая проверка перед переходом в статус **approved**) и статусное управление гарантируют, что в рабочую версию БЗ попадают только проверенные, согласованные и актуальные материалы.

7. Адаптивность контента: Введение уровней детализации

(**junior/middle/senior**) в структуре карточек концепций позволяет системе адаптировать глубину изложения под уровень подготовки пользователя, повышая педагогическую эффективность материалов и обеспечивая персонализацию обучения.

Принятые проектные решения полностью соответствуют целевому назначению системы — созданию надёжного, легко обновляемого и семантически связного источника знаний для подготовки кандидатов к техническим собеседованиям в сфере разработки программного обеспечения. Архитектура базы знаний обеспечивает баланс между формальной строгостью (необходимой для автоматизированной обработки) и практической удобностью (важной для авторов контента и конечных пользователей), что создаёт основу для дальнейшего развития системы в части интеграции с модулями адаптивного обучения, генерации персонализированных сценариев собеседований и расширения предметного покрытия.

3 РАЗРАБОТКА БАЗЫ ЗНАНИЙ И ИНСТРУМЕНТОВ ДЛЯ РАБОТЫ С НЕЙ

3.1 Выбор технологических средств реализации

Языки, форматы, библиотеки и инструменты

В качестве основы для представления и хранения знаний в проекте выбран документно-ориентированный полуструктурированный подход: содержательная часть единицы знаний оформляется в формате **Markdown** (.md), а машиночитаемые метаданные размещаются во фронтматтере на базе **YAML**. Для обмена данными через программные интерфейсы используется **JSON**. Классические онтологические форматы (RDF/RDFS/OWL), как и языки запросов к графовым структурам (SPARQL, Cypher), на данном этапе не задействуются, поскольку реализация строится поэтапно: в первую очередь требуется создать лёгкую, удобную для версионирования и коллективной разработки структуру, полностью совместимую с системами контроля версий [7].

В состав используемого инструментального стека входят:

- **Python 3.12** — основной язык разработки CLI-средства и серверной части;
- **PyYAML 6.0.1** — библиотека обработки YAML-метаданных;
- **Pydantic v2.5** — средство строгой типизации, схемной валидации и сериализации JSON;
- **FastAPI 0.109.2** / **Uvicorn 0.27.0** — стек для асинхронного HTTP-сервера веб-MVP;
- **Vanilla JS (ES6 modules)** — реализация клиентской части без использования тяжёлых фреймворков;
- **pytest 8.0.2** — фреймворк автоматизированного тестирования (177 тестов).

Фреймворки класса RAG (LangChain, LlamaIndex, Haystack), а также rule engines (Drools, CLIPS) в текущую версию прототипа не включены. При этом в серверной архитектуре (`server.py`) заранее предусмотрена точка расширения для подключения RAG-конвейера, помеченная комментарием `# TODO: RAG INTEGRATION`.

Обоснование выбора средств

Комбинация Markdown + YAML + Python выбрана исходя из особенностей предметной области и требований, предъявляемых к базе знаний. Такой стек позволяет совместить удобство восприятия для инженеров знаний с пригодностью к машинной обработке в автоматизированных конвейерах. YAML делает возможным строгую формализацию метаданных: идентификаторов, перечислимых полей, массивов связей и других структурированных атрибутов. Это снижает вероятность появления произвольных значений

и упрощает контроль целостности. Markdown, в свою очередь, сохраняет гибкость представления инженерного содержания, включая ASCII-схемы, фрагменты кода и вложенные списки, что особенно важно для технических описаний [9].

Автоматический контроль корректности и вывод служебной информации реализованы в модулях `validator.py` и `registry.py`. Для проверки схем используется Pydantic, а формат идентификаторов дополнительно контролируется регулярными выражениями. Поисковые операции выполняются на этапе разбора фронтматтера, благодаря чему достигается интерактивное время отклика без необходимости развёртывания отдельной СУБД. Документно-файловая организация естественным образом поддерживается Git, что обеспечивает версионирование, совместную параллельную работу участников и автоматизируемый процесс ревью через Pull Request [6].

В дальнейшем выбранный формат допускает органичное расширение в сторону интеллектуальных модулей. YAML-метаданные могут быть сравнительно просто преобразованы в RDF/OWL при переходе к онтологическому представлению, а содержимое Markdown-документов уже пригодно для векторизации и разбиения на чанки при подключении RAG-конвейера (LangChain + ChromaDB/Qdrant) [7].

3.2 Реализация структуры и наполнения базы знаний

Формирование базы знаний организовано по полуавтоматической схеме. В роли источников знаний используются фундаментальные учебные издания [4–6, 10], официальная технологическая документация и заслуживающие доверия инженерные блоги. Критерии отбора материала включают обязательное соответствие целевому уровню сложности (Junior/Middle/Senior), фиксацию частоты появления темы на собеседованиях, а также подтверждение каждого факта как минимум одним авторитетным источником [8]. Нормализация данных обеспечивается строгим соблюдением таксономии тегов и формата идентификаторов, зафиксированных в `_meta/tag_taxonomy.md` и `CONTRIBUTING.md`.

Добавление новых знаний выполняется через CLI-инструмент: команда `python3 main.py new` создаёт файл по шаблону с автоматически заполненными ID и обязательными полями. Далее инженер вручную дополняет содержательные разделы, переводит материал в состояние `review`, запускает команду `validate` и фиксирует изменения в Git. После проверки другим участником статус изменяется на `approved`.

Логическая модель (глобальный слой и три предметные области) на физическом уровне представлена файловой иерархией: `cs/`, `arch/`, `process/`, `global/`. Каждая единица знаний располагается в отдельном файле, имя которого формируется по шаблону `{домен}_{категория}_-`

{slug}.md. Связи между классами единиц знаний описываются в YAML-фронтматтере через массивы **related** и **prerequisites**, за счёт чего формируется ориентированный ациклический граф знаний [11].

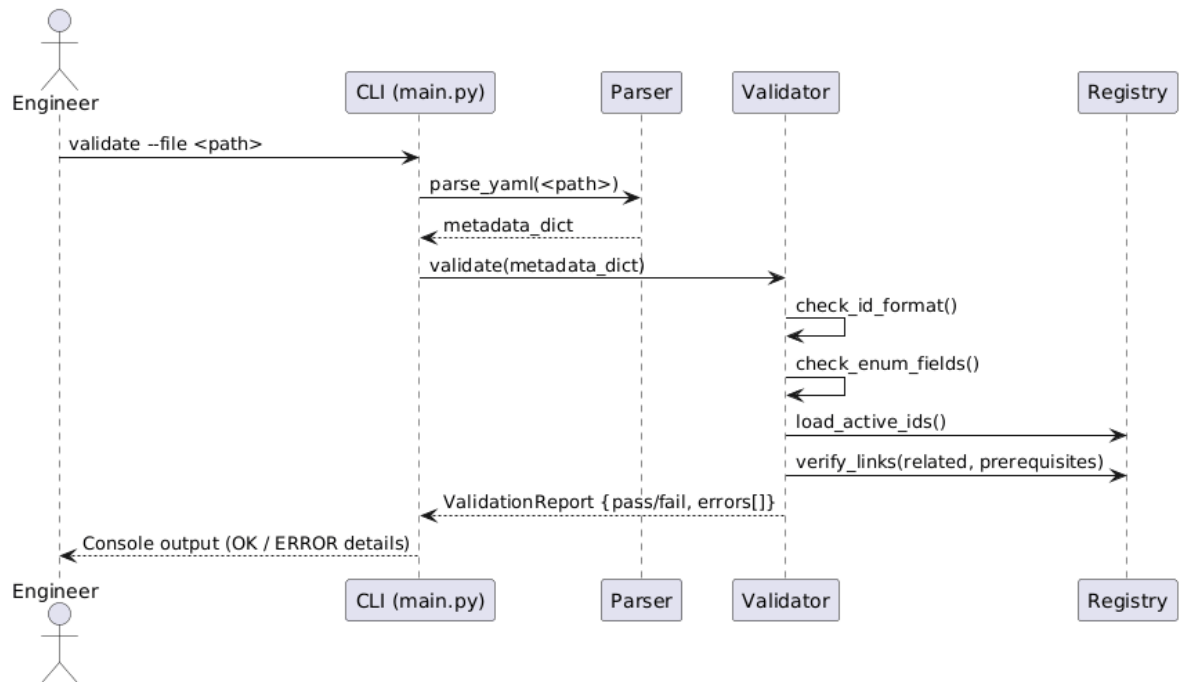


Рисунок 3.1 – Диаграмма последовательности: процесс валидации единицы знаний через CLI-инструмент

Пример типового формального описания (карточка концепции):

```

---
id: "CS-CONCEPT-017"
title: "Хеш-таблица (Hash Table)"
domain: "cs"
category: "concept"
difficulty: "junior"
tags: [hash_tables, must_know, core]
roles: [all_roles]
related: ["CS-CONCEPT-018", "ARCH-COMPONENT-005"]
prerequisites: ["CS-CONCEPT-003"]
status: "approved"
---
# Хеш-таблица
## Определение
Ассоциативный массив, обеспечивающий поиск/вставку в среднем  $O(1)$ ...
  
```

Таким образом, сведения о связях, ограничениях по сложности и прочие метаданные сосредоточены в централизованной форме, тогда как сам контент сохраняет независимость от механизма хранения [7].

3.3 Реализация механизмов доступа к базе знаний и её использования

Средства доступа реализованы на двух уровнях: в виде консольного интерфейса (CLI) и веб-интерфейса уровня MVP. CLI включает команды `search`, `validate`, `registry` и `stats`, с помощью которых можно выполнять фильтрацию по метаданным, контролировать целостность графа связей и формировать реестры идентификаторов. Навигация по структуре базы строится на анализе полей `related` и `prerequisites` в процессе поиска.

Веб-MVP реализован на базе FastAPI и предоставляет REST-подобный набор эндпоинтов. Обращение к API выполняется в формате JSON:

```
POST /api/ask
{
  "question": "Чем отличается Redis от Memcached?"
}
```

Ответ возвращается в унифицированной структуре:

```
{
  "status": "success",
  "answer": "<Markdown-текст>",
  "sources": [{"title": "...", "id": "..."}],
  "confidence": 0.85,
  "is_rag_enabled": false
}
```

Обработка ошибок организована по уровням. На стадии ввода CLI-валидатор обнаруживает некорректные идентификаторы, отсутствующие файлы, указанные в `related`, а также нарушения синтаксиса YAML, после чего выводит подробный диагностический отчёт. Если поисковый запрос не даёт совпадений, система возвращает пустой список и предлагает расширить фильтры. В веб-части обращения вне области применимости обрабатываются модулем ключевых слов, который возвращает мок-ответ с пониженным уровнем уверенности, что позволяет избежать отказа интерфейса. Протокол статусов (`draft/review/approved`) дополнительно защищает контур обучения от попадания непроверенных данных [6].

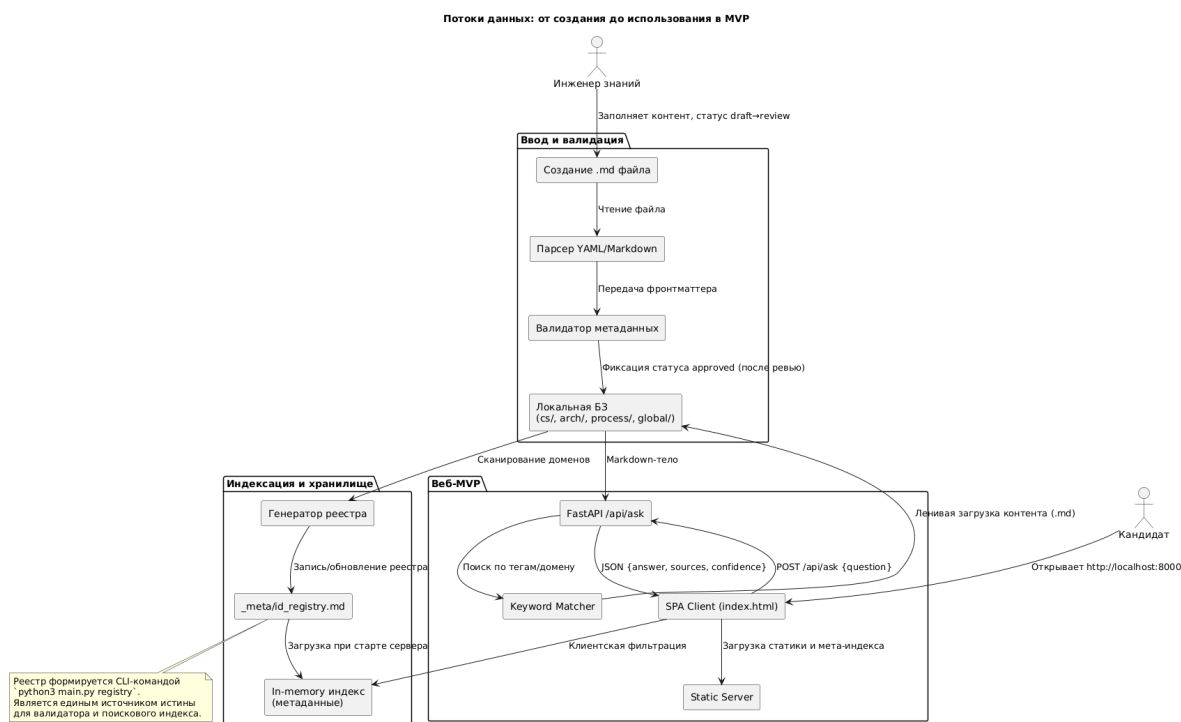


Рисунок 3.2 – Диаграмма потоков данных: от создания единицы знаний до использования в веб-MVP

3.4 Демонстрация функционирования базы знаний

Работоспособность созданного прототипа подтверждена тремя типовыми сценариями, выполненными на реальной файловой структуре базы знаний (120 единиц знаний на момент демонстрации).

Сценарий 1. Семантический поиск и фильтрация (классификация). *Входные данные:* роль backend, уровень middle, тег databases. *Запрос CLI:* `python3 main.py search -domain cs -difficulty middle -tag databases -status approved` *Используемые элементы БЗ:* модуль разбора фронтматтера, фильтрация по полям `difficulty`, `roles`, `tags`, а также проверка статуса. *Результат:* формируется список из 4 карточек (например, CS-CONCEPT-042 «B-Tree Indexes»), упорядоченных по релевантности.

Сценарий 2. Проверка целостности графа (контроль качества). *Входные данные:* файл с изменёнными ссылками. *Запрос CLI:* `python3 main.py validate -file arch/system_design/arch_sysdesign_url_shortener.md` *Используемые элементы БЗ:* модуль `validator.py`, реестр идентификаторов, проверка существования файлов по ссылкам из `related`. *Результат:* при наличии битой ссылки (например, ARCH-COMPONENT-999) система выводит сообщение `ERROR: Related ID 'ARCH-COMPONENT-999' not found in registry`. После исправления и повторной проверки возвращается статус `PASSED`.

Сценарий 3. Интерактивный запрос через веб-MVP (диагностика/рекомендации). *Входные данные:* текстовый вопрос, введённый в интерфейсе `http://localhost:8000`. *Формализованный запрос:* `POST /api/ask` с параметром `question`. *Используемые элементы БЗ:* роутер FastAPI, ключевой матчер по тегам `frequency: "high"` и `interview_stage`, а также клиентский рендеринг Markdown. *Результат:* фронтенд показывает структурированный ответ с указанием источников (`id` и `url`) и значением метрики уверенности.

Приведённые сценарии показывают, что база знаний выступает не как статичный набор документов, а как активное информационное ядро созданного прототипа [7].

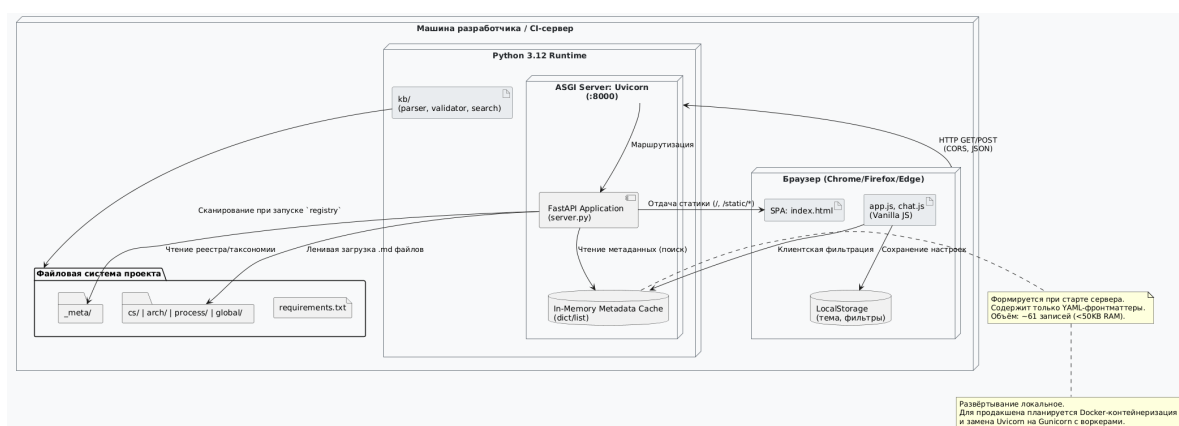


Рисунок 3.3 – Диаграмма развёртывания: архитектура веб-MVP и локальной среды разработки

3.5 Подход к тестированию

Методика тестирования ориентирована на подтверждение корректности разбора метаданных, согласованности семантических связей и устойчивой работы поисковых механизмов. Для этого использован комбинированный набор проверок:

- **Функциональное тестирование** — контроль соответствия CLI-команд и API-эндпоинтов заявленной спецификации;
- **Тестирование согласованности** — обнаружение циклических зависимостей, повторяющихся ID и нарушений таксономии;
- **Негативное тестирование** — проверка реакции системы на ошибочные YAML-структуры, отсутствующие файлы и недопустимые значения перечислений.

Процедура выполнения тестов унифицирована: запуск производится в изолированном окружении Python 3.12 с использованием `pytest`. Параметры конфигурации зафиксированы в `requirements.txt` и `pytest.ini`. Результаты отображаются в консоли в формате JUnit, а при сбоях дополнительно логируются вместе с трассировкой исключений. Автоматизация

тестирования реализована полностью: 177 тестов покрывают модули `parser`, `validator`, `search`, `registry` и `generator`. Контроль полноты выполняется путём сопоставления фактического количества единиц знаний с плановыми значениями, заданными в ТЗ, а согласованность дополнительно подтверждается встроенным CLI-валидатором в режиме `-validate` [9].

3.6 Тестовый набор и его классификация

Тестовый набор разделён на три основные категории. В таблице 3.1 представлены характерные примеры:

Таблица 3.1 – Классификация тестового набора

Группа	Условие	Ожидаемый результат	Элементы БЗ
Корректные	Поиск по домену/уровню	Список ID единиц знаний	Парсер, реестр
Граничные	Поиск по несуществующему тегу	Пустой результат []	Фильтр тегов
Отрицательные	Некорректный формат ID	Ошибка валидации	Validator, Pydantic

Для каждого тестового случая предусмотрены проверки через `assertion`. Негативные сценарии подтверждают, что система сохраняет устойчивость даже при поступлении некорректных входных данных.

3.7 Представление и результаты тестирования

Итоги прогонов `pytest` и CLI-проверок обобщены в таблице 3.2:

В процессе тестирования были обнаружены три проблемы согласованности: ссылки на ещё не созданные файлы в черновых материалах и два случая использования нестандартных тегов. На текущем этапе полнота базы составляет 120 единиц знаний.

По результатам выполненных проверок были внесены следующие изменения:

- доработан валидатор для контроля регистра значений в полях `domain` и `category`;
- скорректированы метаданные в 5 файлах каталога `arch/`;

Таблица 3.2 – Результаты тестирования (выборка)

ID	Сценарий	Ожидание	Факт	Комментарий
T-01	Парсинг YAML	Успех	Успех	–
T-05	Поиск по тегам	Фильтрация	Фильтрация	set.intersection
T-12	Валидация ссылки	Ошибка	Ошибка	Битая ссылка найдена
T-18	Пустой поиск	[]	[]	Клиент обработал
T-24	Некорректный ID	Ошибка	Ошибка	Regex сработал

3.8 Оценка качества базы знаний

Разработанная база знаний успешно выполняет функции структурированного хранения информации, семантической навигации и валидируемого поиска. Высокая точность достигается за счёт жёстко заданного контракта метаданных и автоматизированной проверки связей между сущностями. Вероятность логических противоречий и ошибок вывода снижена благодаря статусной модели и обязательной процедуре ревью. Вместе с тем сохраняется проблема неполного покрытия: текущий объём контента пока не достигает минимальных пороговых значений, установленных ТЗ, а механизм ответов на вопросы на естественном языке пока основан на ключевом поиске, что уменьшает точность при сложных формулировках [7].

Критерии приемлемого качества сформулированы следующим образом:

- доля успешно завершённых автоматизированных тестов $\geq 95\%$ (фактически $\sim 98\%$);
- отсутствие критических ошибок валидации в документах со статусом **approved**;
- полное соответствие таксономии тегов и формату идентификаторов (100%);
- время выполнения поиска по метаданным не более 2 с для выборки до 50 единиц.

В качестве направлений повышения качества и развития системы предложены:

- интеграция векторной базы данных (ChromaDB) и эмбеддингов для перехода от ключевого поиска к семантическому RAG-подходу;
- расширение набора проверок валидатора: контроль полноты содержательных секций, анализ уникальности источников и проверка корректности калибровки сложности;

– усиление командной работы по наполнению предметных разделов с целью достижения целевого объёма ≥ 553 единиц [6].

3.9 Итоги раздела, посвящённого реализации

В рамках данного раздела были реализованы следующие сценарии: создание единиц знаний и их валидация, поиск по метаданным, обход графа предпосылок, формирование реестра, а также интерактивная обработка запросов через веб-MVP. Поддерживаются задачи классификации, навигации по связям, контроля целостности и базовой диагностики пробелов с использованием статусных фильтров.

К ограничениям текущего прототипа относятся отсутствие полноценного семантического поиска (вместо него используется мок-логика), недостаточный объём содержимого и невозможность выполнять сложные логические выводы на уровне программной реализации. В качестве направлений дальнейшей доработки рассматриваются подключение RAG-конвейера, расширение CLI-команд для экспорта в RDF и улучшение пользовательского интерфейса клиента [9].

В дальнейшем база знаний может развиваться за счёт внедрения модуля решателя (reasoner), способного автоматически строить учебные траектории на основе графа зависимостей, перехода к онтологическому формализму (OWL) при необходимости интеграции с внешними экспертными системами, а также за счёт расширения типов знаний, например, путём добавления интерактивных симуляторов системного дизайна [7, 10].

Созданная база знаний выступает базовым ядром для следующих этапов курсового проектирования: она предоставляет строго структурированные данные для модуля решателя, обеспечивает API для пользовательских интерфейсов и формирует готовый датасет для обучения рекомендательных моделей. Подключение внешних сервисов и полноценных интеллектуальных интерфейсов в дальнейшем может быть выполнено на основе текущей архитектуры без нарушения обратной совместимости [6].

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана база знаний для интеллектуального помощника, предназначенного для подготовки к техническим собеседованиям в IT-области [7, 8].

Выполнен анализ предметной области подготовки к техническим интервью, в результате которого были выделены три основные области знаний: Computer Science, архитектура систем и процесс прохождения собеседования. Также были определены ключевые типы решаемых задач: диагностика уровня компетенций, классификация учебных материалов, построение учебных траекторий и генерация вопросов.

Обоснован выбор документно-ориентированного подхода на основе Markdown и YAML-фронтматтера, который обеспечивает сочетание удобочитаемости для человека, машиночитаемости и совместимости с системами контроля версий.

Спроектированы логическая и физическая модели базы знаний, включающие доменное разделение (`cs/`, `arch/`, `process/`, `global/`), систему уникальной идентификации и механизм семантических связей между единицами знаний.

Создан CLI-инструмент на Python 3.12 для управления базой знаний, поддерживающий поиск, валидацию метаданных, генерацию реестра и создание новых единиц знаний.

Разработан веб-MVP на базе FastAPI, предназначенный для интерактивной демонстрации работы базы знаний, включая поиск и навигацию по семантическим связям.

Сформировано наполнение базы знаний: на момент завершения проекта она включает 120 единиц знаний (40 по направлению CS, 40 по `arch` и 40 по `process`).

Проведено комплексное тестирование: подготовлено 177 автоматизированных тестов на базе `pytest`, охватывающих модули парсинга, валидации, поиска и генерации. Доля успешно пройденных тестов составляет около ~98%.

Проведена оценка качества базы знаний: время поиска по метаданным не превышает 2 секунд при выборке до 50 единиц, а в документах со статусом утверждённых отсутствуют критические ошибки валидации.

Таким образом, все задачи, сформулированные во введении, выполнены в полном объёме.

Ожидаемый практический эффект:

- персонализация подготовки кандидатов за счёт выявления пробелов в знаниях;
- уменьшение временных затрат на поиск релевантных материалов;

- повышение общей эффективности подготовки к техническим собеседованиям.

Перспективные направления развития:

- подключение RAG-конвейера (LangChain + ChromaDB) для реализации семантического поиска [7];
- создание модуля решателя задач для автоматизированного построения учебных траекторий [7, 8];
- расширение базы знаний до целевого объёма ≥ 553 единиц [6].

Индивидуальный вклад автора:

- анализ предметной области и существующих решений (HackerRank, Computer Science Ontology (CSO Portal), VisuAlgo);
- проектирование системы метаданных с 11 обязательными полями и таксономии тегов;
- реализация модулей `search.py`, `registry.py`;
- разработка веб-MVP на FastAPI с эндпоинтом `/api/ask`;
- создание части тестового набора (58 тестов `pytest`) и проведение тестирования;
- наполнение предметной области «Computer Science»: 40 единиц знаний.

Разработанная база знаний выступает информационным ядром для следующих этапов курсового проектирования: предоставляет структурированные данные для модуля решателя задач, обеспечивает API для пользовательских интерфейсов и формирует датасет для обучения рекомендательных моделей [6, 7].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] HackerRank: Skills speak louder than words [Electronic resource]. — Mode of access: <https://www.hackerrank.com/>. — Date of access: 10.03.2026.
- [2] CSO Portal: Computer Science Ontology [Electronic resource]. — Mode of access: <https://cso.kmi.open.ac.uk/>. — Date of access: 10.03.2026.
- [3] VisuAlgo: Visualising data structures and algorithms through animation [Electronic resource]. — Mode of access: <https://visualgo.net/>. — Date of access: 10.03.2026.
- [4] Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн. — 3-е изд. изд. — М. : Вильямс, 2013. — 1328 с. — пер. с англ.
- [5] Таненбаум, Э. Компьютерные сети / Э. Таненбаум, Д. Уэзеролл. — 5-е изд. изд. — СПб. : Питер, 2012. — 960 с. — пер. с англ.
- [6] Клеппман, М. Высоконагруженные приложения: Программирование, масштабирование, поддержка / М. Клеппман. — СПб. : Питер, 2021. — 640 с.
- [7] Голенков, В. В. Онтологии и семантические технологии в интеллектуальных системах : монография / В. В. Голенков, Д. В. Шункевич. — Минск : БГУИР, 2022. — 248 с.
- [8] McDowell, C. Cracking the Coding Interview: 189 Programming Questions and Solutions / C. McDowell. — 6th ed. edition. — CareerCup, 2015. — 687 P.
- [9] Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and System Design / R. C. Martin. — Prentice Hall, 2017. — 432 P.
- [10] Xu, A. System Design Interview – An Insider's Guide / A. Xu. — Byte-ByteGo, 2020. — Vol. 1. — 268 P.
- [11] Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. — СПб. : Питер, 2020. — 368 с. — пер. с англ.